WHITEPAPER

# Verifiable charity on the XRP Ledger

Proof comes before payment.

**Brianne Salato**
Founder and Product Architect

Version 0.6 (Draft) · 20 October 2026
verichar.net · VeriChar@proton.me
X: @VeriCharOnChain

# Contents

1. What is live today
2. Executive summary
3. The problem
4. The VeriChar approach
5. System architecture
6. Cryptographic proof, explained
7. Donor and charity experience
8. Planned modules
9. Business model
10. Security and risk
11. Compliance considerations
12. Roadmap
13. Conclusion
Appendix A. Sources

**Document status.** This is Version 0.6, a draft. It replaces the public link that still pointed at a missing Version 0.2 file, and it updates Version 0.5 so the paper matches the product as it actually stands on 20 October 2026. VeriChar is a working prototype. The public website and its four-step demonstration are simulations. They do not connect a wallet, submit an XRP Ledger transaction, verify a charity, accept a proof upload, or move money. A Testnet-only API foundation exists. It does not submit transactions and it does not touch Mainnet. The flow described in later sections is the intended design. It will be tested on the XRP Ledger test network, and reviewed by legal and security professionals, before any real funds are handled. Statements about how the XRP Ledger works were checked against XRPL.org and Ripple RLUSD documentation and are tagged [S1] to [S24]. Design details marked “to be confirmed” are not yet verified against the XRPL specification. This document is not legal, financial, tax, or investment advice.

## 1. What is live today

Version 0.6 exists so a reader of verichar.net and a reader of this paper see the same product. The following is built. Everything else in this document is a plan.

| Surface | Now | Not yet |
|---|---|---|
| Public site | Mission, proposed flow, transparency limits, and business model. Four-step simulation and an Enterprise Portal preview. | No donations, charity verification, published needs, proof uploads, or vendor payments. |
| Simulation | Sample need moves from listed, to locked, to proof submitted, to released. | No wallet, XRPL transaction, verification, proof upload, or payment. |
| Testnet API | Public health check. Authenticated tenant context and validated Testnet account lookup. Bearer tokens are stored only as SHA-256 hashes, bound to one tenant and explicit scopes. The public page may show whether the health check is reachable. The browser is not given a token. | No saved tenant records, transaction submission, escrow, oracle, OAuth, or Mainnet. |
| Charity interest | Email VeriChar@proton.me. Verification will be free when it opens. | Registration is closed. No IRS lookup. No organization is shown as verified. |

| Surface | Now | Not yet |
|---|---|---|
| Enterprise Portal | Preview of corporate impact reporting, with sample rows labeled as samples. | Not certified. Not legal or tax advice. Not a live reporting product. |

## 2. Executive summary

Charitable giving runs on trust, and trust is hard to verify. Donors rarely see what their gift actually bought. Charities carry the burden of proving impact after the money is already spent. Fraud, mismanagement, and vague “administrative costs” thrive in that gap.

VeriChar (Verified Charity On-Chain) is a verification layer for charitable giving, built on the XRP Ledger (XRPL). The intended design is specific. Every donation funds one itemized need and is paid in RLUSD, a USD stablecoin on the XRPL. The funds lock in escrow on the ledger and release directly to the vendor only when two proofs are verified: an itemized merchant invoice and proof of delivery. The charity never holds the cash. Every settlement step is written to a public ledger that anyone can audit.

That design is not operating with real money. VeriChar does not currently process real donations.

VeriChar is free for donors and charities. Planned revenue is a 1% vendor settlement fee and licensing of audit-ready corporate social responsibility (CSR) data to enterprise customers. How the fee is collected at escrow release is still an open design question. Section 9 records the working proposal and what still has to be proven on Testnet.

> **The core principle.** Proof comes before payment, not after. Trust should be enforced by code, not demanded by slogans. The code that would enforce it is not finished. Until it is, the public demonstration says so on the screen.

## 3. The problem

Today’s giving systems ask donors to take organizations at their word. Four structural problems follow.

**Opacity.** Donations are usually pooled into general funds. A donor cannot trace a specific dollar to a specific purchase.

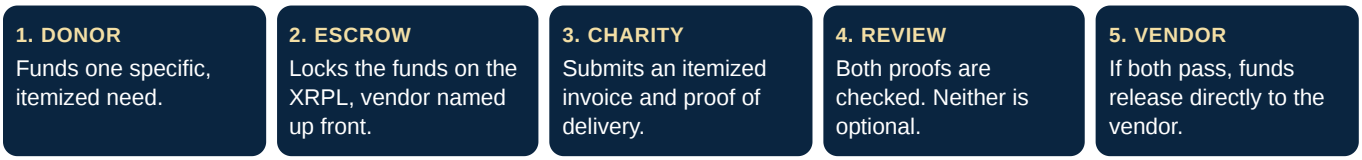
**Diversion and fraud.** Unverified organizations, misused funds, and inflated costs are difficult to detect. When money moves first and reports follow later, problems surface only after the damage is done.

**The burden of proof.** Honest charities must assemble reports and receipts to demonstrate impact. The process is slow, manual, and hard for an outsider to audit.

**Privacy versus transparency.** Many donors want to give without being tracked, yet still want proof that their gift was used properly. Existing platforms usually force a choice between the two.

## 4. The VeriChar approach

VeriChar changes the order of events. Instead of paying first and reporting later, funds would be locked first and released only when the evidence exists. The sequence below is the intended flow. The public simulation shows these steps with sample data and stops there.




- **Verified charities, when verification exists.** The intended check binds an organization to its EIN and to a wallet address. The XRPL supports a two-way link between an account and a domain: the account’s Domain field plus an

`xrp-ledger.toml` file hosted at that domain [S12]. The EIN check itself happens off-ledger. XRPL Credentials are another possible fit: an issuer creates a credential for a subject account, and the subject must accept it [S19] (to be confirmed for VeriChar). None of this runs on the website today. There is no IRS lookup, and no charity is marked verified.

- **Itemized needs.** A charity would list a specific need with a price and a named vendor, for example “4 cases of water, $20.” Donors would fund that item, not a general pool. Need publishing is not live.
- **Escrow.** The donation would lock in an escrow on the XRP Ledger. An escrow locks XRP or fungible tokens, which cannot be used or destroyed until conditions are met [S17]. The escrow names its recipient, here the vendor, when it is created, and finishing it delivers the funds to that recipient [S17]. VeriChar does not yet create escrows.
- **Two proofs.** The charity would submit an itemized invoice and proof of delivery. Delivery proof may be a time-stamped, GPS-checked photo, a carrier delivery confirmation, or a signed receipt, depending on the item. The site does not accept uploads.
- **Direct settlement.** When both proofs pass, the escrow would be finished and the funds would go straight to the vendor. The charity would not touch the money. Vendor payment is not live.

# 5. System architecture

VeriChar has four layers, each with a single job. Only a thin slice of the application layer and none of the ledger settlement layer are built.

### Ledger layer: the XRP Ledger

VeriChar plans to use the XRPL’s native escrow feature rather than custom smart-contract code, which is intended to keep the attack surface small. The XRPL is a decentralized blockchain [S1]. A new ledger version is created when 80% of validators agree on a set of transactions, a consensus process the documentation says takes 4 to 6 seconds [S1]. Transaction costs are small amounts of XRP quoted in drops, where 1 million drops equal 1 XRP [S9]. The documentation’s example transactions use 10 to 12 drops [S8]. Speed matters for emergency orders that must reach merchants quickly. Cost matters most for small donations, as covered under costs of escrow below.

Escrow is a native XRPL payment type that locks XRP or fungible tokens until conditions are met, with an automated system in the ledger replacing the traditional third party [S17]. The XRPL supports three types: time-based, conditional, and combination escrow [S17]. VeriChar’s design uses a conditional escrow. A cryptographic condition acts as the lock, and the matching fulfillment releases the funds [S17]. The lifecycle is:

- **Create.** The sender submits an EscrowCreate transaction that defines the amount, the release conditions, and the recipient. For XRP escrows this can include a finish-after time, a crypto-condition, and an optional expiry [S17]. A conditional escrow with no finish-after time must have an expiry (CancelAfter) [S17]. Since the fix1571 amendment, EscrowCreate must include a Condition or a FinishAfter [S20].
- **Finish.** An EscrowFinish transaction delivers the funds to the recipient if the conditions are met [S17]. In a conditional escrow anyone can finish it, but only by supplying the correct fulfillment [S17].
- **Expire and cancel.** If nobody finishes the escrow before its expiry, it becomes expired and can no longer be finished. Anyone can then cancel it with EscrowCancel, which returns the funds to the sender [S17]. An escrow with no CancelAfter field never expires and cannot be canceled [S17].

**What this means for VeriChar.** Every VeriChar escrow must carry an expiry, or a failed delivery could lock funds permanently. The fulfillment must be guarded, because whoever holds it can trigger release [S17]. The only crypto-condition type the XRPL supports is PREIMAGE-SHA-256 [S17]. Timed releases and expirations follow ledger close times and can vary by about five seconds, and escrows cannot be created with past time values [S17]. A Smart Escrows amendment, which would let developers program release rules with WebAssembly, is listed as in development, so VeriChar does not depend on it [S20].

**Costs of escrow.** The XRPL escrow documentation lists limits that matter to a charity platform [S17]. Escrow needs two transactions, one to create it and one to finish or cancel it. Crypto-conditions cost more than ordinary transactions, and the documentation says these costs can make escrow infeasible for small amounts. While an escrow is incomplete, the sender is responsible for its reserve. An EscrowFinish with a fulfillment must pay at least 330 drops plus 10 drops per 16 bytes of fulfillment, based on a 10-drop reference cost, and the formula scales if fee voting changes the reference fee. Multi-signing adds its own cost on top [S17]. VeriChar must measure these costs on the test network against real gift sizes such as “4 cases of water, $20,” before any minimum gift size is set.

## Ledger facts that shape the design

- **Finality.** A transaction result is final only when it appears in a validated ledger [S8]. VeriChar will count only validated results, check for the result code `tesSUCCESS`, and read `meta.delivered_amount` instead of the Amount field, which can be exploited through partial payments [S15]. The Testnet account lookup already requests `ledger_index: validated`.
- **Reserves.** Every XRPL account must hold a base reserve (1 XRP on Mainnet), and each escrow an account owns adds an owner reserve (0.2 XRP on Mainnet) [S10]. Reserve values can change by validator vote [S10]. While an escrow is incomplete, the sender is responsible for its reserve [S17], so the design must decide which account sends each escrow and how that reserve is funded. A Sponsor amendment, which would let others cover fees and reserves for users, was not enabled in the amendments snapshot used for Version 0.5 [S20]. VeriChar does not rely on it. Recheck the amendments page before any launch.
- **Accounts and keys.** There is no create-account transaction. A Payment of enough XRP to a valid address creates the account, and whoever holds the secret key has full control of it [S4]. An account can be authorized by a master key, a rotatable regular key, or a multi-signing signer list, and the master key can be kept offline [S4, S14]. Defaults differ between tools: many client libraries such as xrpl.js default to Ed25519, while `wallet_propose` defaults to secp256k1, so one seed can give a different address unless the algorithm is set explicitly [S18]. The ripple-keypairs library also defaults to ed25519 [S16]. An account cannot send transactions until it is funded, and no administrator can reverse an applied transaction [S18]. No seed or private key is stored in the public site or in the API token configuration. Tokens are hashes only.
- **Test networks.** Testnet and Devnet let developers create accounts funded with test XRP [S1]. The current API connects only to the official Testnet WebSocket endpoint, `wss://s.altnet.rippletest.net:51233`. It does not accept an arbitrary server URL from a caller.
- **Servers.** Serious use is better served by running our own rippled and Clio servers, because a malicious server could misreport payments [S6]. The prototype uses a public Testnet server and is not a production deployment.
- **RLUSD.** RLUSD (Ripple USD) is a USD stablecoin issued as a native XRPL token. An issuer holds real-world assets as collateral and creates equivalent tokens on the ledger [S22]. The Mainnet issuer address is `rMxCKbEDwqr76QuheSUMdEGf4B9xJ8m5De` and the Testnet issuer is `rQhWct2fv4Vc4KRjRgMrxa8xPN9Zx9iLKV` [S22]. An account must hold a trust line to the issuer before it can receive RLUSD [S22]. Ripple states that one RLUSD trust line takes at least 1.2 XRP (1 XRP base reserve plus 0.2 XRP owner reserve) [S22]. The XRPL Reserves page describes an exception for the first two trust lines [S10], so this must be confirmed on the test network. XRP is still needed for fees and reserves [S9, S10].
- **Escrowing RLUSD.** RLUSD is a trust-line token. Trust-line tokens can be escrowed only if the issuer enables Allow Trust Line Locking, and token escrows must always have an expiry [S17]. TokenEscrow showed as enabled on 2026-02-12 in the snapshot reviewed for Version 0.5 [S20]. The RLUSD issuer settings table lists clawback as enabled, Require Auth and No Freeze as not enabled, and does not list the locking flag [S23]. Whether RLUSD can be escrowed is therefore not confirmed. An RLUSD EscrowCreate must be tested against the Testnet issuer before any further build depends on it. If authorization is not required, missing trust lines may be auto-created for escrow recipients [S17]. Transfer rates are captured at creation and apply at completion [S17], and the settings table does not list an RLUSD transfer rate [S23]. A deep-frozen holder cannot finish an escrow but can still cancel one [S17].

- **Credentials.** The Credentials amendment showed as enabled on 2025-09-04 [S20]. An issuer creates a credential for a subject account, the subject must accept it to make it valid, and either party can delete it [S19]. A credential is identified by its issuer, subject, and credential type [S19]. VeriChar could issue a verified-charity credential after an off-ledger review (to be confirmed). The website does not issue credentials.

### Verification layer: the dual check

An “oracle,” in this paper, means a service that checks real-world facts and then releases a fulfillment. It is not the XRPL’s own on-ledger Oracle object, which carries price data [S10]. VeriChar’s intended service runs two checks that must both pass. It is not built.

- **Supplier log (invoice).** Optical character recognition reads the merchant invoice and compares the vendor, line items, and pricing with the funded need.
- **Delivery log (proof of delivery).** For photos, the service inspects embedded metadata (EXIF), a cryptographic timestamp, and the GPS location. For shipped goods, it can instead accept a carrier delivery confirmation. A signed provider record can stand in for a service, such as a clinic visit.

When both checks pass, the service would release the fulfillment that unlocks the escrow [S17]. Because the recipient was named when the escrow was created, finishing the escrow delivers the funds to that recipient [S17]. The public simulation shows these states as labels. It does not verify an invoice or a photo.

### Privacy layer

The intended product lets donors give as guests with no VeriChar account. An XRPL address is still needed to send funds, because only signed transactions from an account can be submitted [S4, S8]. An account that sends RLUSD needs an RLUSD trust line [S22] and XRP for fees and reserves [S9, S10], so how guest wallets are created and funded is an open design question. A donor who wants a tax receipt may provide an email address, which would be encrypted and stored off-chain, never written to the ledger. XRPL transactions are public [S15]. A “stealth escrow” idea is planned to reduce the visible link between a donor’s wallet and a specific donation. Because the XRPL is a public ledger, that idea needs careful engineering and legal review, and it is not built. The current site collects no donor email and stores no personal data.

### Application layer

Three interfaces are planned: a donor view for browsing verified charities and funding needs, a charity view for onboarding and submitting proof, and an enterprise view for CSR reporting. Today the public site is a single page with those sections as explanations, a simulation, an email link for charity interest, and an Enterprise Portal preview. The Testnet API is a separate service. It is not wired into the simulation, and the page does not hold a bearer token.

## 6. Cryptographic proof, explained

“Cryptographic proof” sounds abstract. Here is what the words mean, and what VeriChar is not claiming.

| Concept | What it means | Why it matters |
|---|---|---|
| Digital fingerprint (hash) | A record is converted into a short, unique code. Change any detail and the code changes completely. Every signed XRPL transaction has a unique hash [S8]. | Tampering is easy to detect. |
| Digital signature | The record is signed with a private key, like a tamper-evident seal. A signed XRPL transaction is immutable, and its signature is not valid for any other transaction [S8]. XRPL key pairs use secp256k1 or ed25519 [S18]. The ripple-keypairs library can sign | Shows who confirmed a record, and when, if that signature scheme is the one we ship. |

| Concept | What it means | Why it matters |
|---|---|---|
| | and verify arbitrary hex-encoded messages [S16]. Whether VeriChar signs delivery records this way is to be confirmed. | |
| Public ledger | Transactions in validated ledgers can be looked up by anyone using their hash [S3, S8]. Which VeriChar proof data is written on-ledger is to be confirmed. The simulation writes nothing. | Anyone can verify what was actually recorded. Recorded data cannot be altered retroactively unless a majority of validators agree [S1]. |

Proof depends on the item. A pallet of supplies may be proven by a carrier confirmation. A dental appointment may be proven by a signed provider record. The rule is constant: in the intended design, funds release only after the proof is verified.

> **Honest limit.** Cryptographic proof shows that a record is authentic and unaltered. It cannot prove that the underlying event was honest. A real photo of the wrong goods, or a real invoice for a padded price, can still be signed. VeriChar is meant to make fraud harder and easier to spot. It cannot make fraud impossible. The live site says this in the same words.

## 7. Donor and charity experience

These are the intended experiences. They are not available on the site, except as the labeled simulation and an email link.

### Donors, when giving is live

- Browse organizations that have passed verification, and search by keyword, state, or cause.
- Fund one itemized need from a specific list, not a general pot.
- Give as a guest. An optional email for a tax receipt would be encrypted off-chain.
- Watch the status move from funds locked, to proof submitted, to released, against validated ledger results rather than a private dashboard alone.

### Charities, when registration is open

- Submit an organization profile, a wallet address, and tax identification for review. Supporting-document requirements are still being designed.
- Only after a future approval process should a charity be shown as verified and allowed to publish itemized needs.
- List needs with a price and a named vendor.
- Submit the invoice and proof of delivery when goods arrive.
- Pay nothing. VeriChar is free for charities.

Registration is not open. Interest goes to VeriChar@proton.me. The site does not look up an EIN, does not store a charity profile, and does not display a verified badge.

## 8. Planned modules

These modules are not in the prototype. They stay on the roadmap only after the basic escrow path is proven on Testnet.

**Automated subscription escrows.** For baseline essentials such as hygiene products, diapers, or non-perishable inventory, a donor could lock a pool of assets into a pre-authorized vault. On a set cycle the vault would create a new

itemized escrow from the shelter's updated inventory. Each release would still require the same two proofs before the vendor is paid. Because the sender covers the reserve of each incomplete escrow [S17, S10], the design must account for reserves across the cycle.

**Micro-donation aggregation.** Small leftover balances can be swept into pooled vaults tied to a cause the donor chooses. The funds would stay locked and move into an itemized escrow only when a full funding goal is met, with the same verification before vendor payment. The XRPL escrow documentation warns that escrow costs can make it infeasible for small amounts [S17], so pooling until a full goal is met is meant to spread those costs across one escrow. Owner reserves [S10] must not swamp very small balances. Whether this is economical has to be measured, not assumed.

## 9. Business model

VeriChar is free for donors and charities. That sentence is already true of the prototype, because the prototype charges nobody and moves no money. The revenue design below is not implemented. How a 1% vendor settlement fee would be collected at escrow release is still an open design question. The working proposal uses only features described in the documentation reviewed. Anything the sources do not confirm stays marked open.

| Revenue idea | Working proposal | Status |
|---|---|---|
| Vendor settlement fee (1%) | For each funded item, the sending account would create two conditional escrows in RLUSD: 99% of the listed price to the vendor and 1% to a VeriChar revenue account. An escrow names its amount, release conditions, and recipient when it is created, and finishing it delivers the funds to that recipient [S17]. Both fulfillments would be released only after both proofs verify. If proof never arrives, both expire and can be canceled, returning the funds to the sender [S17]. A $20 item would split into 19.80 and 0.20 RLUSD, assuming 1 RLUSD equals 1 USD, which the pages reviewed do not state as a protocol guarantee. Two escrows are proposed because one escrow has one named recipient [S17]. | Open. RLUSD escrow is not confirmed (Section 5). Also open: whether one condition can lock both escrows; whether Batch, shown as enabled on 2026-10-09 in the Version 0.5 snapshot [S20], can bundle two EscrowCreate transactions; and whether two escrows make small gifts too expensive [S17, S10]. Taking a share of donor-funded escrows belongs in the money-transmission review in Section 11. |
| Issuer transfer fee | An issuer can charge a percentage when users send its token. The fee is debited from the sender, only the destination amount is credited, and the fee leaves the circulating supply, which benefits the issuer [S24]. | Not VeriChar revenue. The RLUSD issuer is Ripple. The settings table reviewed does not list a transfer fee [S23]. Confirm with `account_info` on Testnet [S13] before relying on a zero fee. |
| CSR reporting license | An enterprise customer would pay a VeriChar revenue account, preferably with an RLUSD Payment after that account has a trust line [S22]. A destination tag is the usual way to tell customers apart, and the RequireDest flag forces one on every incoming payment [S12]. A subscription would be credited only after a validated transaction with result `tesSUCCESS` and `meta.delivered_amount` [S15]. What would be licensed is the verification, matching, and reporting built on public ledger records, not the records themselves. Those records are already public [S8, S3, S15]. | Not built. The Enterprise Portal on the website is a preview with sample numbers. Invoicing, fiat billing, and converting RLUSD to bank currency sit outside the XRPL. Checks are another on-ledger option [S20]. Payment Channels are described for XRP [S20]; their use with RLUSD is not covered by the sources. |

A revenue account, when one exists, would be funded with at least the base reserve [S4, S10]. Adding an RLUSD trust line raises the XRP that must be held [S22]. The master key should stay offline, day-to-day signing should use a regular key [S18], and withdrawals should require multi-signing [S11]. Every transaction burns an XRP fee [S9, S10]. Revenue in RLUSD and costs in XRP means VeriChar must hold an XRP float. The sources give no dollar value for XRP, so

margins cannot be computed from this paper. They have to be measured on Testnet. None of these accounts exist for production use.

## 10. Security and risk

No system removes risk. These are the principal risks the intended design is meant to address. Mitigations marked as planned are not in the prototype.

| Risk | Planned mitigation |
|---|---|
| Forged or inflated invoices | Automated parsing, price checks against the listed need, duplicate detection, a verified vendor registry, and human review of exceptions. |
| Photo or GPS spoofing | Metadata checks combined with signed capture, cross-checks against carrier or vendor confirmations, and random audits. |
| Collusion between charity and vendor | Vendor verification, price benchmarking, anomaly detection, and a channel for reporting concerns. |
| Verifier compromise or error | Hardened key management, an auditable log, and a later move toward independent verifiers that must agree. The XRPL supports multi-signing: a signer list of 1 to 32 addresses with weights and a quorum [S11]. Anyone holding a conditional escrow's fulfillment can finish it [S17], so fulfillment secrets must be guarded like keys. Whether multi-signing can gate release is to be confirmed. The prototype holds no fulfillment secrets. |
| Lost or stolen keys | Whoever holds an account's secret key has full control of it [S4]. Keep master keys offline, sign with a rotatable regular key [S14, S4], and use multi-signing for high-value accounts [S11]. |
| Misreading payments | Count only validated transactions with result `tesSUCCESS`, and use `delivered_amount` rather than the Amount field [S15, S8]. Token amounts need more precision than the XRP examples in the tutorial [S15]. |
| Reserve shortfalls | Each escrow adds an owner reserve, and the sender covers it while the escrow is incomplete [S10, S17]. Fund reserves explicitly and monitor with `account_info`, which reports OwnerCount [S13, S10]. |
| Stuck funds if delivery fails | Set CancelAfter on every escrow. After it passes, the escrow cannot be finished and anyone can cancel it, returning the funds to the sender [S17]. Without CancelAfter an escrow never expires and cannot be canceled [S17]. |
| Cost on small gifts | Measure real costs on the test network and set a minimum item size, or pool gifts, if the numbers say escrow is too expensive [S17]. |
| Key algorithm mismatch | Set the algorithm explicitly in every tool and test address derivation [S18]. |
| Untrusted servers | Run our own rippled and Clio servers before production use [S6]. The prototype uses a public Testnet server. |
| Asset value and XRP float | Gifts are planned in RLUSD so the gift itself is not an XRP price bet [S22]. XRP is still required for fees and reserves [S9, S10]. Hold only the float those costs require. |
| Issuer actions on | The issuer controls the token. The settings table reviewed lists clawback as enabled and No Freeze as not enabled [S23], so the issuer can recover tokens or freeze them [S12, S20]. Freezes do not prevent finishing a trust-line token |

| Risk | Planned mitigation |
|---|---|
| RLUSD | escrow, but a deep-frozen holder cannot finish one and can only cancel [S17]. Whether clawback reaches escrowed RLUSD is not covered by the sources. Confirm the issuer address in Ripple’s documentation before any launch [S22]. |
| Privacy leaks | Data minimization, off-chain encryption, and an independent privacy review before launch. All XRPL transactions are public [S15]. The current site stores no donor profiles. |
| A reader mistaking the demo for a live till | The website labels the demonstration as a simulation on the demo itself and in the footer. This paper repeats that limit in the status box and in the roadmap. |

## 11. Compliance considerations

Handling charitable funds involves regulation that varies by state and by activity. The following areas require review by qualified legal counsel before any live launch. Nothing here is that review.

- **Money transmission.** Holding or moving customer funds may trigger federal (FinCEN) registration and state money-transmitter licensing. The escrow design aims to avoid VeriChar taking custody. A 1% fee taken from donor-funded escrows may change that analysis. Both points must be confirmed. The prototype takes no custody because it moves no funds.
- **Charitable solicitation.** Many states require registration for entities that solicit donations on behalf of charities. The prototype does not solicit or accept donations.
- **Anti-money-laundering.** Vendors and larger donors may need screening, even where small guest gifts remain private.
- **Tax treatment.** Digital assets are treated as property for US tax purposes. Charities, not VeriChar, would issue tax receipts.
- **Data protection.** Encrypted email and any personal data must follow applicable privacy law. The current site does not collect that data.
- **Token and securities rules.** The XRPL documentation warns that some token offerings may be regulated as securities in the United States and advises researching the relevant regulations before engaging in any financial service business [S9]. VeriChar is not offering a token.

## 12. Roadmap

| Phase | Focus | Status on 20 October 2026 |
|---|---|---|
| 0. Public prototype | Public website, four-step simulation, charity-interest email, and an Enterprise Portal preview with sample data. The simulation is labeled. No wallet, no ledger transaction, no verification, no proof upload, and no payment. | Live at verichar.net |
| 0.5 Testnet API foundation | Health check, hashed bearer tokens bound to a tenant and scopes, authenticated tenant context, and validated Testnet account lookup. Official Testnet endpoint only. | In the repository. Not yet persistent storage, transaction submission, OAuth, escrow, oracle, or Mainnet. |
| 1. Test network | Confirm that Testnet RLUSD can be escrowed [S22, S17]. Then test create, finish, and cancel; measure costs on small gifts; settle reserves, key algorithms, and the still-open | Planned. Not started as ledger transactions. |

| Phase | Focus | Status on 20 October 2026 |
|---|---|---|
| settlement | 1% collection design. Try a charity-verification credential [S19]. Build the invoice and delivery checks. No real funds. Testnet RLUSD has no financial value [S22]. | |
| 2. Pilot | One to three partner charities and a small set of vendors, with small amounts, only after legal and security review. | Planned |
| 3. Expansion | Subscription escrows, pooled small gifts, more than one independent verifier, and a CSR product that is no longer sample data. | Planned |

# 13. Conclusion

Generosity should not require surrendering common sense. VeriChar's rule is simple: proof before payment, using ledger features that already exist. Donors should be able to see what a gift bought. Honest charities should be able to prove their work. Vendors should be paid directly. The record should belong to everyone.

That rule is not yet enforced by code. Version 0.6 says so in the same place it describes the design. The public site says so on the simulation, in the transparency section, and in the footer. The next honest step is the Testnet work in phase 1, not a claim that the escrow already pays vendors.

### Trust should be enforced by code, not demanded by slogans.

# Appendix A. Sources

Technical statements about the XRP Ledger in this document were checked against the pages below for Version 0.5 and carried forward here. Recheck them before launch. Reserves, fees, and amendments change. Amendment statuses cited from [S20] are as of the page snapshot reviewed for Version 0.5, not a fresh crawl on 20 October 2026. [S22] and [S23] are Ripple documentation for RLUSD, not XRPL.org. Product-status statements in Sections 1, 7, 9, and 12 describe the VeriChar repository and verichar.net, not the XRP Ledger.


1. [S1] What is the XRP Ledger? https://xrpl.org/docs/introduction/what-is-the-xrp-ledger
2. [S2] Concepts. https://xrpl.org/docs/concepts
3. [S3] Ledgers. https://xrpl.org/docs/concepts/ledgers
4. [S4] Accounts. https://xrpl.org/docs/concepts/accounts
5. [S5] Payment types. https://xrpl.org/docs/concepts/payment-types
6. [S6] Networks and servers. https://xrpl.org/docs/concepts/networks-and-servers
7. [S7] Consensus protocol. https://xrpl.org/docs/concepts/consensus-protocol
8. [S8] Transactions. https://xrpl.org/docs/concepts/transactions
9. [S9] Tokens. https://xrpl.org/docs/concepts/tokens
0. [S10] Reserves. https://xrpl.org/docs/concepts/accounts/reserves
1. [S11] Multi-signing. https://xrpl.org/docs/concepts/accounts/multi-signing
2. [S12] AccountSet. https://xrpl.org/docs/references/protocol/transactions/types/accountset
3. [S13] account_info. https://xrpl.org/docs/references/http-websocket-apis/public-api-methods/account-methods/account_info
4. [S14] wallet_propose. https://xrpl.org/docs/references/http-websocket-apis/admin-api-methods/key-generation-methods/wallet_propose
5. [S15] Monitor incoming payments with WebSocket. https://xrpl.org/docs/tutorials/advanced-developer-topics/client-library-development/monitor-incoming-payments-with-websocket
6. [S16] ripple-keypairs. https://github.com/XRPLF/xrpl.js/tree/main/packages/ripple-keypairs
7. [S17] Escrow. https://xrpl.org/docs/concepts/payment-types/escrow
8. [S18] Cryptographic keys. https://xrpl.org/docs/concepts/accounts/cryptographic-keys
9. [S19] Manage credentials. https://xrpl.org/docs/tutorials/compliance-features/manage-credentials
0. [S20] Known amendments. https://xrpl.org/resources/known-amendments

1. [S21] Credential issuing service code sample. https://github.com/XRPLF/xrpl-dev-portal/tree/master/_code-samples/issue-credentials/js/
2. [S22] RLUSD on the XRP Ledger. https://docs.ripple.com/products/stablecoin/developer-resources/rlusd-on-the-xrpl
3. [S23] RLUSD XRP Ledger settings. https://github.com/ripple/RLUSD-Implementation/blob/main/doc/rlusd-xrpl-settings.md
4. [S24] Stablecoin configuration. https://xrpl.org/docs/concepts/tokens/fungible-tokens/stablecoins/configuration

**About the author.** Brianne Salato, Founder and Product Architect of VeriChar. Contact: VeriChar@proton.me · X: @VeriCharOnChain · verichar.net

---

Disclaimer. This document is for information only. It is not legal, financial, tax, or investment advice. It describes a product still in development. Some features may change or may not be built. Statements about future plans are goals, not guarantees. Nothing in this document is an offer to sell or a solicitation to buy any asset or security. VeriChar does not currently process real donations. Version 0.6 (Draft), 20 October 2026.